M. Frisbie, *Building Browser Extensions*
https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-8725-5_5

5. The Extension Manifest

Matt Frisbie[1]
(1) California, CA, USA

The extension manifest is the blueprint of the browser extension. Broadly speaking, it is a config file containing a collection of key-value pairs that dictate what the extension can do and in what fashion.

The content of a manifest differs between manifest versions and browsers. Not all properties can be used by all browsers; some browsers will partially support a property, or even not support it at all. Between manifest v2 to manifest v3, new properties are added, others are removed; and some change in how they are defined.

MDN offers a very good table of manifest property support here: https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/manifest.json#browser_compatibility.

> **Note** This chapter omits a handful of manifest properties which are either not widely supported by most web browsers (such as `theme` and `theme_experiment`), or are completely undocumented (such as `natively_connectable`).

The Manifest File

The extension manifest is defined in a single `manifest.json` file that lives in the root directory of the extension. This is the only file that is required for a browser extension to be considered valid:

```
simple-extension-directory/
└─ manifest.json
```

> **Note** This file is a standard JSON file, with the notable exception that `//`-style comments are allowed. If your text editor is using a standard JSON linter, it may indicate that comments are not allowed in `manifest.json`—ignore this error. All browsers and extension stores allow comments in the manifest.

The JSON file contains a single object with a large number of required and optional properties that configure how the extension behaves. There are only three required properties:

- `manifest_version`, which indicates to the browser how the manifest should be interpreted. This has a large number of implications on the

behavior of the extension.

- `version`, which declares the version number of the extension. This is used by extension stores for differentiating between releases.
- `name`, which declares the name of the extension. This is used by both the browser as well as extension stores as the top-line identifier of the extension.

Therefore, the simplest possible manifest would be the following:

manifest.json

```
{
  "name": "MVP Extension",
  "version": "1.0",
  "manifest_version": 3
}
```

Supporting Different Locales

Several manifest properties define strings and textual images that are visible to the user. In situations where the extension needs to support multiple languages based on the user's locale, hardcoding these values is not appropriate. To account for this, the manifest supports loading strings from message files based on the user's locale.

To add locale support, message files must be added to a `_locales` directory. This directory contains one directory for each locale you wish to support. Inside each of these subdirectories, a single `messages.json` file defines the localized strings. The manifest can then automatically load these strings with the special `__MSG_messagename__` identifier.

The following example shows a simple manifest that supports English and French locales:

```
localized-extension-directory/
├─ manifest.json
└─ _locales/
   ├─ en/
   │  └─ messages.json
   └─ fr/
      └─ messages.json
```

The manifest file would be written as follows:

manifest.json

```
{
  "name": "__MSG_extensionName__",
  "version": "1.0",
  "manifest_version": 3,
  "default_locale": "en",
}
```

When using locales, the `default_locale` property is required in your manifest. The required formatting of the `messages.json` files is shown below:

_locales/en/messages.json

```
{
  "extensionName": {
    "message": "Hello world!"
  }
}
```

_locales/fr/messages.json

```
{
  "extensionName": {
    "message": "Bonjour le monde!"
  }
}
```

Note Locales and internationalization is a broad and deep subject. Refer to **https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/Internationalization** for additional details on locale placeholders, additional message properties, and more.

Note The WebExtensions i18n API and extension CSS files can also use locale strings from these `messages.json` files. Refer to the *Extension and Browser APIs* chapter for details.

Match Patterns and Globs

Several properties in the manifest can use match patterns and globs to specify multiple URLs or files at once.

File Path Match Patterns

Properties such as `web_accessible_resources` can use match patterns to select single files or entire directories. As is the case with all file paths in the manifest file, they are resolved with respect to the root directory of the extension. The syntax allows for the use of `*` wildcards to match zero to many characters. The following are some examples of valid file match patterns:

- `"/*"` selects all files
- `"/foo/*"` selects all files in the `foo` directory
- `"/foo/*.png"` selects all PNG files in the `foo` directory
- `"/foo/bar.png"` selects a single PNG file in the `foo` directory

URL Match Patterns

Properties such as `content_scripts` can use match patterns to whitelist single URLs or entire web origins. The syntax allows for the use of `*` wildcards to match

zero to many characters. The following are some examples of valid URL match patterns:

- "<all_urls>" selects all URLs with a protocol of http://, https://, ws://, wss://, ftp://, data://, or file://
- "https://*/*" selects all URLs with a https:// protocol
- "*://example.com/*" selects all URLs with an example.com origin
- "https://example.com/foo/*" selects all example.com URLs with a foo/ path prefix
- "https://example.com/foo/bar" selects a single URL

Note For specific details on the rules for match patterns, refer to https://developer.chrome.com/docs/extensions/mv3/match_patterns/.

URL Globs

Properties such as content_scripts can use globs to add additional filtering to URL match patterns. Globs are an extension of URL match patterns; their syntax allows for the use of * wildcards to match zero to many characters, as well as ? to match exactly one character:

- "*://example.com/*" selects all URLs with an example.com origin
- "*://example.???/*" selects all URLs with an example. origin followed by a three-character TLD, such as example.com, example.org and example.gov

Note For specific details on the rules for globs, refer to https://developer.chrome.com/docs/extensions/mv3/content_scripts/#matchAndGlob

Manifest Properties

This section contains a comprehensive list of manifest properties and how their values should be defined. Many of these properties are linked to additional browser permissions or parts of the WebExtensions API; each section will direct you to the portion of the book detailing the interplay of the manifest property value, required permissions, and the involved APIs.

If you wish to inspect the browser source code involved with parsing manifest properties, use the following resources:

- Chromium manifests: https://chromium.googlesource.com/chromium/src/+/main/chrome/common/extensions/api/_manifes
- Firefox manifests: https://searchfox.org/mozilla-central/source/browser/components/extensions/schemas

Tip To experiment with these properties in a real extension, you can add them to the "MVP Extension" manifest.json defined at the beginning of the chapter. (Many of these properties require supplemental APIs, permissions, and files to work.) Recall that each change to the manifest requires an extension reload.

action

This property defines an object containing values which dictate the behavior and appearance of the extension icon in the browser toolbar (Figure 5-1). This is the manifest v3 replacement for the `browser_action` and `page_action` properties in manifest v2.

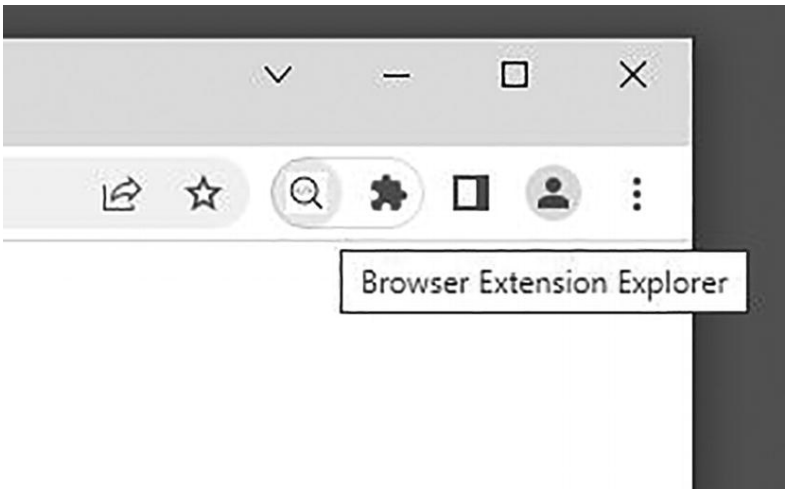

Figure 5-1 The extension toolbar icon with hover title revealed

Note This property can be only used in manifest v3. If writing for manifest v2, use `browser_action`.

The property with is shown below with a fully defined example value:

action property in manifest.json

```
{
  ...
  "action": {
    "default_icon": {
      "16": "icons/icon-16.png",
      "32": "icons/icon-32.png",
      "64": "icons/icon-64.png"
    },
    "default_popup": "popup/popup.html",
    "default_title": "Building Browser Extensions",
    "browser_style": true,
    "theme_icons": [
      {
        "light": "icons/icon-16-light.png",
        "dark": "icons/icon-16.png",
        "size": 16
      },
      {
        "light": "icons/icon-32-light.png",
        "dark": "icons/icon-32.png",
        "size": 32
      }
    ]
  },
  ...
}
```

The `action` object allows for the following properties:

default_icon

This property tells the browser where to locate icons for the toolbar via relative paths to image files. There are two ways to define this property: either provide PNG or JPEG image icons and allow the browser to intelligently select which icon it would prefer, or provide a single vector image.

Providing one or multiple image icons allows the browser to decide which icon is best for the current hardware configuration. (For example, an Apple Retina display prefers an icon with twice the resolution.) Providing icons of 16x16px, 32x32px, and 64x64px is sufficient to cover all possible cases. The following is an example of how this property can be used:

Example value for default_icon with sized images

```
{
  ...
  "action": {
    "default_icon": {
      "16": "icons/icon-16.png",
      "32": "icons/icon-32.png",
      "64": "icons/icon-64.png"
    },
    ...
  },
  ...
}
```

If you wish to provide a vector instead, the browser only needs a single vector file to account for all rendering configurations:

Example value for default_icon with vector image

```
{
  ...
  "action": {
    "default_icon": "icons/icon.svg"
    ...
  },
  ...
}
```

default_popup

This property tells the browser where to locate the HTML file that should render as a popup when the extension's toolbar icon is clicked. This can also be set or changed via the WebExtension API's `chrome.action.setPopup()` method. The following is an example of how this property can be used:

Example value for default_popup

```
{
  ...
  "action": {
    "default_popup": "components/popup/popup.html"
    ...
  },
  ...
}
```

Note If this property is not defined, no popup page will appear when the extension's toolbar icon is clicked. Instead, the browser will dispatch a click event in any background scripts (if they exist).

default_title

This property tells the browser the text that should render as a tooltip when the extension's toolbar icon is hovered over. This can also be set or changed via the WebExtension API's `chrome.action.setTitle()` method. The following is an example of how this property can be used:

Example value for default_title

```
{
  ...
  "action": {
    "default_title": "Open Explorer menu"
    ...
  },
  ...
}
```

Note If this property is not defined, the browser will use the extension `name` property as the hover text.

browser_style

(Firefox only) This property is a boolean that tells the browser if it should inject a stylesheet in the popup to style it consistently with the browser. These stylesheets can be viewed in any Firefox browser at `chrome://browser/content/extension.css`, or `chrome://browser/content/extension-mac.css` on macOS. The property defaults to `false`. The following is an example of how this property can be used:

Example value for browser_style

```
{
  ...
  "action": {
    "browser_style": true
```

```
    ...
  },
  ...
}
```

theme_icons

(Firefox only) This property allows you to define alternate icons based on the current active Firefox theme. It effectively extends the behavior of `default_icon`. The following is an example of how this property can be used:

Example value for theme_icons

```
{
  ...
  "action": {
    ...,
    "theme_icons": [
      {
        "light": "icons/icon-16-light.png",
        "dark": "icons/icon-16.png",
        "size": 16
      },
      {
        "light": "icons/icon-32-light.png",
        "dark": "icons/icon-32.png",
        "size": 32
      }
    ],
    ...
  },
  ...
}
```

author

This property provides an author name for display in the browser. The following is an example of how this property can be used:

Example value for author

```
{
  ...
  "author": "Matt Frisbie"
  ...
}
```

> **Note** If the `developer.name` property is defined, it will override this value.

automation

(Chromium browsers only) This property allows developers to access the accessibility tree for the browser. This property is only needed in cases where the extension is directly targeting assistive devices. Per MDN, "Browsers create an accessibility tree based on the DOM tree, which is used by platform-specific Accessibility APIs to provide a representation that can be understood by assistive technologies, such as screen readers."

> **Note** Technologies supporting accessibility are out of the scope of this book. For details on using this API, refer to https://developer.chrome.com/docs/extensions/reference/automation/ and https://developer.mozilla.org/en-US/docs/Glossary/Accessibility_tree.

background

This property allows you to define what file or files will be used for the background script. This property is used in both manifest v2 and v3, but the value of the property has significant differences between the two versions.

Manifest v2 background scripts

One way of creating background scripts in manifest v2 was to provide references to one or more script files in a `scripts` array. These scripts are loaded in their array order. They are loaded in a headless web page and will have access to a window object and the DOM. The following is an example of how to provide background scripts in this fashion:

Example value for background scripts

```
{
  ...
  "background": {
    "scripts": [
      "scripts/background-1.js",
      "scripts/background-2.js"
    ]
  },
  ...
}
```

Manifest v2 background page

Another way of creating background scripts in manifest v2 was to provide a `page` reference to a single HTML file with `<script>` tags that load the background script files. This page is rendered as a headless web page, and therefore these scripts are loaded in whatever order the browser's JavaScript engine performs. The following is an example of how to provide background scripts in this fashion:

Example value for background page

```
{
  ...
  "background": {
    "page": "background.html"
  },
  ...
}
```

This strategy has several advantages. It allows the user to use import statements by adding `type="module"`, as well as to add content to the HTML headless page.

Manifest v2 persistent pages and event pages

In addition to defining how background scripts should be loaded, the background property also allows you to define how the scripts should execute via the boolean `persistent` property. The default value is `true`:

- When `persistent` is `true`, a persistent background page is created. It is kept in memory at all times. It starts up when the extension is loaded or the browser is launched, and it lasts until the extension is unloaded or the browser is closed.
- When `persistent` is `false`, a nonpersistent background page is created. Sometimes referred to as an "event page," this background script can be unloaded and reloaded whenever the browser decides it is idle.

The following is an example of how to provide background scripts in this fashion:

Example value for persistent background page

```
{
  ...
  "background": {
    "page": "background.html",
    "persistent": true
  },
  ...
}
```

Manifest v3 service worker

In manifest v3, the options for creating a background script are much more limited. Because persistent pages in manifest v2 had the potential to consume too many system resources, all background scripts for manifest v3 must execute as a service worker. The closest analog of this in manifest v2 is a single script event page.

Example value for background service worker

```
{
  ...
  "background": {
```

```
    "service_worker": "scripts/background.js"
  },
  ...
}
```

In order to enable using `import` statements in the background script, `"type": "module"` can be added:

Example value for background service worker with a module script

```
{
  ...
  "background": {
    "service_worker": "scripts/background.js",
    "type": "module",
  },
  ...
}
```

> **Note** Refer to the *Background Scripts* chapter for details on setting up background scripts.

browser_action

(Manifest v2 only) The `browser_action` property was replaced by `action` in manifest v3. The property values and behaviors are effectively identical. If using manifest v2, refer to the `action` property for details on how to assign a value for this property.

> **Note** This property is an artifact of extension tooling for older browser versions. `browser_action` and `page_action` are redundant in all modern browsers. As a result, these properties have been merged into the `action` property, which is identical in format and behavior.

browser_specific_settings

This property defines values that are relevant only to certain browsers. It is rare to have the need to use this property at all. The property takes the form of an object with keys that allocate values to particular browser vendors. Common keys are `edge`, `gecko`, and `safari`, which correspond to browsers matching those vendors/engines. The following is an example of how this property can be used:

Example value for browser_specific_settings

```
{
  ...
  "browser_specific_settings": {
    "edge": {
      "browser_action_next_to_addressbar": false
    },
    "gecko": {
```

```
      "id": "owner@example.com",
      "strict_min_version": "51.0",
      "strict_max_version": "62.*",
      "update_url": "https://example.com/updates.json"
    },
    "safari": {
      "strict_min_version": "21",
      "strict_max_version": "28"
    }
  },
  ...
}
```

Note Google Chrome does not support this key.

chrome_settings_overrides

This property allows the extension to override the default settings for certain browser interfaces. Its value is an object with the following optional properties:

- `homepage` overrides the default homepage of the browser
- `search_provider` defines a new search provider to add to the browser
- `startup_pages` overrides the default start pages of the browser. This is only supported in Chromium browsers

The browser will indicate that these settings are applied in the extension management interface (Figure 5-2).

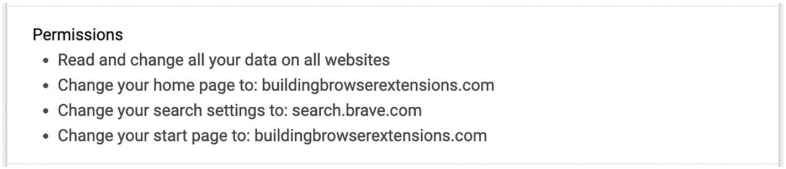

Figure 5-2 Google Chrome's extension management interface showing the permissions for an extension with applied settings overrides

Custom Homepage

The browser homepage is the URL that the browser directs the user to when they click the "home" button in their browser (Figure 5-3). All major browsers no longer display a home button by default, but it can be easily enabled in the browser's settings menu. The homepage URL can be overridden as follows:

Figure 5-3 Google Chrome with homepage button enabled and showing the result of clicking the home-page button

Example value for overriding homepage

```
{
  ...
  "chrome_settings_overrides": {
    "homepage": "https://www.buildingbrowserextensions.com"
  },
```

```
  ...
}
```

Custom Search Engine

The search provider decides where to send queries typed into the browser's address bar. Extensions can access the query string via `searchTerms`, interpolate it into a URL of choice, and dispatch that URL (Figures 5-4, 5-5, 5-6, 5-7).

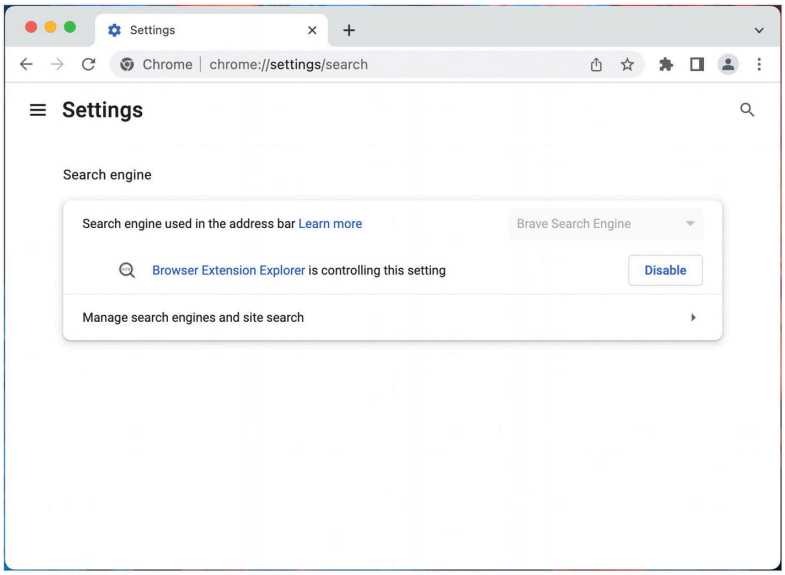

Figure 5-4 Chrome settings page indicating an extension is controlling the default search engine

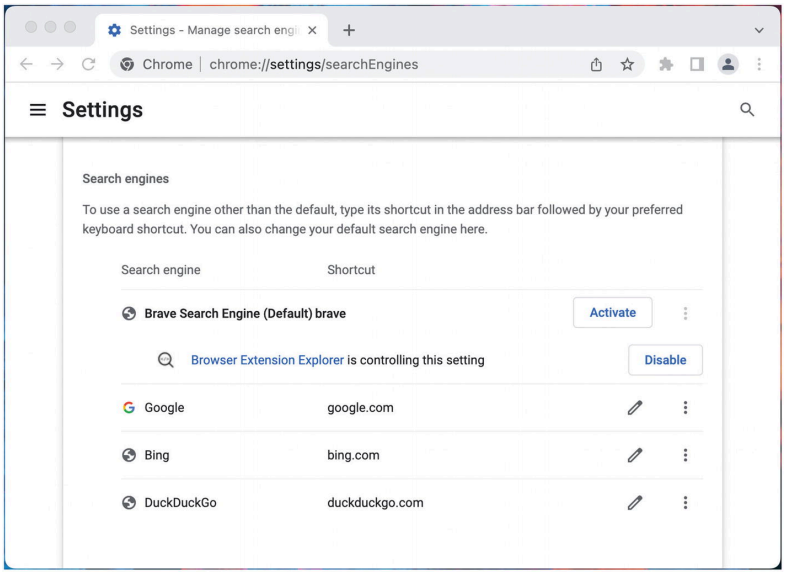

Figure 5-5 Chrome search engine settings showing an extension controlling the default engine

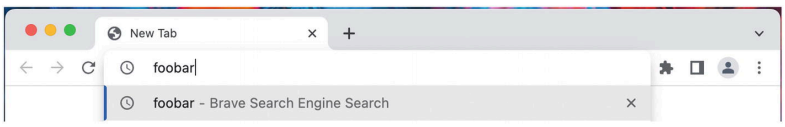

Figure 5-6 Typing in a search query into the address bar shows it being routed to the custom search engine

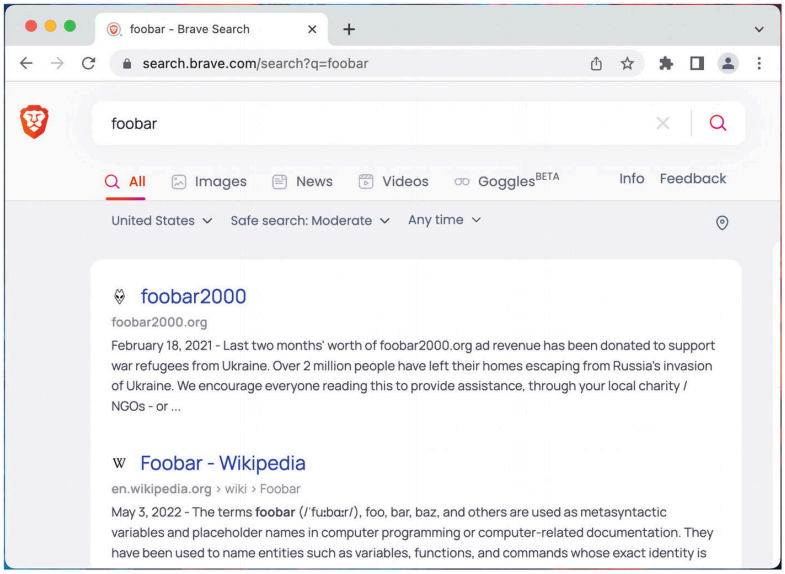

Figure 5-7 Results after executing the address bar search

NoteThe browser's search engine settings are initialized early in the browser startup. You may need to reboot your browser entirely to see these manifest changes take effect.

The search engine can be overridden as follows:

Example value for overriding search_provider

```
{
  ...
  "chrome_settings_overrides": {
    "search_provider": {
      "name": "Brave Search Engine",
      "search_url": "https://search.brave.com/search?q={searchTerms}",
      "encoding": "UTF-8",
      "is_default": true
    }
  }  ...
}
```

The `search_provider` object can also define the following supplementary properties:

- `alternate_urls`
- `favicon_url`
- `image_url`
- `image_url_post_params`
- `instant_url`
- `instant_url_post_params`
- `keyword`
- `prepopulated_id`
- `search_url_post_params`
- `suggest_url`
- `suggest_url_post_params`

Note Support and documentation of these supplementary properties is limited – your mileage may vary. For details, refer to https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/manifest.json/chrome_settings_overrides.

Custom Startup Page

(Chromium browsers only) The startup page is the page that opens when the browser program initially starts up. An extension can set the startup page by providing an array containing exactly one URL (Figure 5-8). The startup page URL can be overridden as follows:

Figure 5-8 Chrome settings displaying the overridden startup page

Example value for overriding startup page

```
{
  ...
  "chrome_settings_overrides": {
    "startup_pages": [
      https://www.buildingbrowserextensions.com
    ]
  },
  ...
}
```

Note The `chrome_` prefix is unrelated to browser compatibility. This property is at least partially supported by Chromium browsers and Firefox.

chrome_url_overrides

This property allows the extension to override the default page for certain browser interfaces. Each extension may override exactly one of the following browser pages:

- History page
- Bookmarks page
- New Tab page

The `chrome_url_overrides` value should be an object with only one of the following properties: `history`, `bookmarks`, `newtab`. The following is an example of how this property can be used:

Example value for chrome_url_overrides

```
{
  ...
  "chrome_url_overrides": {
    "newtab": "components/newtab/newtab.html"
  },
  ...
```

```
}
```

Note The `chrome_` prefix is unrelated to browser compatibility. This property is at least partially supported by Chromium browsers and Firefox.

commands

This property is used to map keyboard commands to perform various tasks in your extension. An example of this is opening and closing the extension popup when the user presses `Ctrl+Shift+F`. Whereas mapping multi-key commands in JavaScript involves setting listeners for `keydown` or similar events, the `commands` property allows you to natively map multi-key keyboard commands using a simple and intuitive syntax. You can define two types of commands: shortcuts, which associate a keyboard command with a predefined set of native browser behaviors, or custom commands, which associate a keyboard command with a special event that can be handled via the WebExtensions API.

Command Syntax

All commands have the same syntax. Each takes the form of an object with two properties: a `suggested_key` object, which defines the multi-key shortcuts that should invoke the command, and a `description` string, which will appear in the extension keyboard shortcut management UI. The following is an example of how to define a shortcut for a generic `foobar` command:

Example value for a generic foobar command

```
{
  ...
"commands": {
    "foobar": {
      "suggested_key": {
        "default": "Ctrl+Shift+F"
      },
      "description": "Run 'foobar' on the current page."
    },  ...
}
```

Once the extension is loaded, the browser will reflect this shortcut in its keyboard shortcut management UI (Figure 5-9).

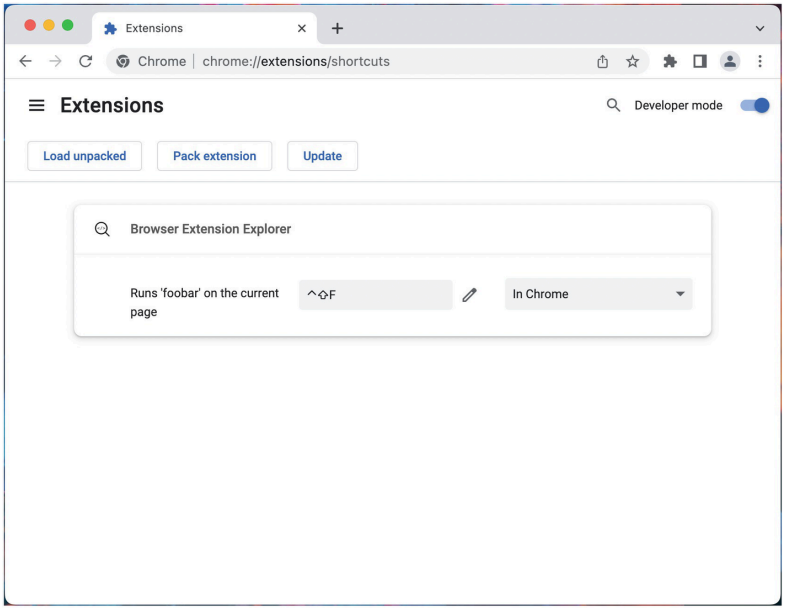

Figure 5-9 Chrome shortcuts page showing the registered extension command shortcut

Note The shortcuts list is available at `chrome://extensions/shortcuts` in Chromium browsers and in the `about:addons` interface in Firefox.

Defining Key Shortcuts

When a key shortcut is defined, the browser will watch for the specified key combination and execute the matching command when they are pressed. The following key identifiers are available for use:

- Alphanumeric Keys: `A-Z` and `0-9`
- General Keys: `Comma`, `Period`, `Home`, `End`, `PageUp`, `PageDown`, `Space`, `Insert`, `Delete`
- Function Keys: `F1-F12`
- Arrow Keys: `Up`, `Down`, `Left`, `Right`
- Media Keys: `MediaNextTrack`, `MediaPlayPause`, `MediaPrevTrack`, `MediaStop`
- Modifier Keys: `Ctrl`, `Alt`, `Shift`, `MacCtrl` (macOS only), `Command` (macOS only), `Search` (ChromeOS only)

Key shortcuts have the following restrictions:

- Key shortcuts cannot overlap with default browser key combinations. For example, `Ctrl+R` is already mapped by the browser to reload the page and therefore is unavailable for use by extension commands.
- Key shortcuts must consist of either two or three keys.
- Key shortcuts must include either `Ctrl` (or `MacCtrl` on MacOS) or `Alt` but not both.

The shortcut is defined as a string with the key identifiers concatenated by `+`. Some examples of valid key shortcuts

- `Ctrl+Shift+F`

- `Alt+Q`
- `MacCtrl+Shift+F12`
- `Alt+MediaPlayPause`

Multi-browser Support

The `suggested_key` object at minimum should define a `default` key shortcut. To support different operating systems, the following properties can additionally be defined:

- `mac`
- `linux`
- `windows`
- `chromeos`
- `android`
- `ios`

If these values are provided and match the host OS, the browser will automatically select them instead of the default value.

Reserved Commands

The browser will give some command names special treatment:

- In manifest v3, `_execute_action` will act as a click on the extension toolbar button.
- In manifest v2, `_execute_page_action` will act as a click on the extension page action button.
- In manifest v2, `_execute_browser_action` will act as a click on the extension browser action button.
- In Firefox and manifest v2, `_execute_sidebar_action` will open the extension's sidebar.

The following command would open the popup when the user uses the `Ctrl+Shift+F` shortcut:

Example value for a reserved _execute_action command

```
{
  ...
  "commands": {
    "_execute_action": {
      "suggested_key": {
        "default": "Ctrl+Shift+F"
      }
    }
  },
  ...
}
```

Note When these reserved commands execute, the browser will not fire the `onCommand()` method.

Custom Commands

For all command identifiers that do not match the reserved commands, the key shortcut will dispatch a command event to all listeners of `command.onCommand()`.

Example value for a generic foobar command

```
{
  ...
  "commands": {
    "foobar": {
      "suggested_key": {
        "default": "Ctrl+Shift+F"
      },
      "description": "Run 'foobar' on the current page."
    }
  },
  ...
}
```

The `foobar` command from above would be handled as follows:

```
chrome.commands.onCommand.addListener((command) => {
  console.log(`Command: ${command}`); // Command: foobar
});
```

Global Commands

By default, when the browser is not focused on the device, keyboard shortcuts will not dispatch any extension commands. Chromium browsers support an optional `global` property for commands which allows commands which use only shortcuts matching `Ctrl+Shift+[0..9]` to be triggered even when the browser does not have focus. The following is an example of this:

Example value for a global foobar command

```
{
  ...
  "commands": {
    "foobar": {
    "suggested_key": {
      "default": "Ctrl+Shift+0"
    },
    "description": "Run 'foobar' on the current page.",
    "global": true
  },  ...
}
```

> **Note** For details on using the commands API, refer to the *Extension and Browser APIs* chapter.

content_capabilities

This property is obsolete and should not be used. Before the release of the Clipboard API and the Persistent Storage API, it was useful for granting extensions the ability to access the clipboard or unlimited storage. With access to these new APIs, this property is no longer needed.

content_scripts

This property defines which files should be injected as content scripts, where to inject them, and in what fashion. Its value is an array of objects, each of which defines a separate set of content scripts and rules for those scripts. Each object contains the following:

- The `matches` property must contain an array of one or more URL matchers. Each URL that matches these URL matchers will have the scripts injected. This property is required.
- The `js` and `css` properties each contain an array of one or more files that should be injected as content scripts when the page URL is a match. One or both should be defined.
- The `run_at` property defines when the content scripts are to be injected. This can be either `document_start`, `document_end`, or `document_idle`. The default value is `document_idle`.
  - `document_start` injects the scripts when `document.readyState === "loading"`
  - `document_end` injects the scripts when `document.readyState === "interactive"`
  - `document_idle` injects the scripts when `document.readyState === "complete"`
- You can optionally subject the URL to three additional filters, evaluated as follows:
  - The `include_globs` property can be defined as a whitelist of URL globs. If provided, the content scripts will *only* be injected if the URL matches this glob array.
  - The `exclude_matches` property can be defined as a blacklist of URL matchers. If provided, the content scripts will *only* be injected if the URL *does not match* this matchers array.
  - The `exclude_globs` property can be defined as a blacklist of URL globs. If provided, the content scripts will *only* be injected if the URL *does not match* this glob array.
  - The `match_about_blank` property enables the content script to be injected on the `about:blank or about:srcdoc` URLs.
  - The `all_frames` property enables the content script to be injected into nested frames.

The following is an example of how this property can be used:

Example value for content_scripts

```
{
  ...
  "content_scripts": [
    {
      "matches": [
        "*://example.com/*"
      ],
      "include_globs": [
        "*archive*"
      ],
      "exclude_matches": [
        "*://example.com/experimental/*"
      ],
      "exclude_globs": [
        "*?display=legacy*"
      ],
      "js": [
        "scripts/content-script.js"
      ],
      "css": [
        "styles/content-script.css"
      ],
      "run_at": "document_end",
      "match_about_blank": true,
      "all_frames": true
    }
  ],
  ...
}
```

Tip The `match_about_blank` key has some very interesting background: https://stackoverflow.com/questions/41408936

Note For additional details on content scripts, refer to the *Content Scripts* chapter.

content_security_policy

This property defines the content security policies (CSP) of the extension. This property was significantly altered between manifest v2 and v3, including in structure and in allowed values.

Manifest v2 content security policy

The manifest v2 `content_security_policy` property was a string that set the CSP values for the entire application. Manifest v2 was very relaxed about CSP restrictions: it freely allowed for script execution such as `eval()` which had the potential to open up security holes. The following is an example of how this property can be used in manifest v2:

Example value for content_security_policy in manifest v2

```
{
  ...
  "content_security_policy": "script-src 'self' 'unsafe-eval' https://example.com; object-src 'self'
  ...
}
```

Manifest v3 content security policy

Manifest v3 considerably changes the `content_security_policy` format. The property is now an object, with two possible keys:

- `extension_pages` defines the CSP for all non-sandboxed extension pages
- `sandbox` defines the CSP for all sandboxed extension pages

Furthermore, manifest v3 in Chromium browsers disallows some CSP values in non-sandboxed extension pages to prevent insecure operations like `eval()` and third-party script execution. The following is an example of how this property can be used in manifest v3:

Example value for content_security_policy in manifest v3

```
{
  ...
  "content_security_policy": {
    "extension_pages": "script-src 'self'",
    "sandbox": "script-src 'self' https://example.com; object-src 'self'"
  },
  ...
}
```

> **Note** Details of extension content security policy definition is covered in the Extension Architecture chapter. For details on manifest v3 CSP restrictions, refer to https://developer.chrome.com/docs/extensions/mv3/intro/mv3-migration/#content-security-policy

converted_from_user_script

This property was used as part of Google's supported transition from userscripts in 2016. Its purpose was so that Chrome Apps, which shared APIs with web extensions, could direct their users to a replacement Progressive Web Application via a `installReplacementWebApp()` method. No modern extensions will have a use for this property.

> **Note** Read more about the transition from Chrome Apps here: https://developer.chrome.com/docs/apps/migration/.

cross_origin_embedder_policy

(Chromium browsers only, manifest v3 only) This property defines the Cross-Origin-Embedder-Policy (COEP) header value for requests to the extension's origin. Because the extension behaves in many ways like a web server, it can be vulnerable to the same origin-based security issues that web servers are. If you wish to enable features that are only accessible with cross-origin isolation, this property is used to set the COEP header value to behave in such a manner. This property must be set in conjunction with the `cross_origin_opener_policy` property to enable cross-origin isolation.

The property contains an object with a single `value` property. The following is an example of how this property can be used to enable cross-origin isolation:

Example value for cross_origin_embedder_policy

```
{
  ...
  "cross_origin_embedder_policy": {
    "value": "require-corp"
  },
  ...
}
```

Note Cross-origin isolation is an important subject outside the scope of this book. For an excellent blog post on it, visit **https://web.dev/coop-coep/**.

cross_origin_opener_policy

(Chromium browsers only, manifest v3 only) This property defines the Cross-Origin-Opener-Policy (COOP) header value for requests to the extension's origin. Because the extension behaves in many ways like a web server, it can be vulnerable to the same origin-based security issues that web servers are. If you wish to enable features that are only accessible with cross-origin isolation, this property is used to set the COOP header value to behave in such a manner. This property must be set in conjunction with the `cross_origin_embedder_policy` property to enable cross-origin isolation.

The property contains an object with a single `value` property. The following is an example of how this property can be used to enable cross-origin isolation:

Example value for cross_origin_opener_policy

```
{
  ...
  "cross_origin_opener_policy": {
    "value": "same-origin"
  },
```

```
  ...
}
```

Note Cross-origin isolation is an important subject outside the scope of this book. For an excellent blog post on it, visit https://web.dev/coop-coep/.

declarative_net_request

(Manifest v3 and Chromium browsers only) This property defines the rulesets to be used for the Declarative Net Request API. The value is an object with a single `rule_resources` key, which defines an array of all the ruleset objects provided in the extension. Each ruleset object must define a unique `id` string, an `enabled` boolean controlling if the browser should use the ruleset, and `path` string with relative path to the ruleset JSON file. The following is an example of how this property can be used:

Example value for declarative_net_request

```
{
  ...
  "declarative_net_request": {
    "rule_resources" : [
      {
        "id": "ruleset_1",
        "enabled": true,
        "path": "ruleset_1.json"
      }, {
        "id": "ruleset_2",
        "enabled": false,
        "path": "ruleset_1.json"
      }
    ]
  },
  ...
}
```

Note For details on the Declarative Net Request API, ruleset definition, and permissions for `declarative_net_request`, refer to the *Networking* chapter.

default_locale

This property defines the default locale string. The property must be present if and only if there is a `_locales` directory as described in the locales section earlier in this chapter. The following is an example of how this property can be used:

Example value for default_locale

```
{
  ...
```

```
  "default_locale": "en",
  ...
}
```

description

This property defines the formal description of the extension. This value is displayed in the browser as well as in the extension marketplaces. The following is an example of how this property can be used:

Example value for description

```
{
  ...
  "description": "A collection of browser extension examples",
  ...
}
```

developer

This property defines an object containing an author name and/or extension homepage URL for display in the browser. Both the `name` and `url` properties are optional. The following is an example of how this property can be used:

Example value for developer

```
{
  ...
  "developer": {
    "name": "Matt Frisbie",
    "url": "https://www.buildingbrowserextensions.com"
  },
  ...
}
```

Note If the `developer` properties are defined, they will override the `author` and `homepage_url` properties.

devtools_page

This property tells the browser where to locate the HTML file that should be used as the entrypoint to your extension's developer tools content. This page is used to bootstrap the developer utilities, but it renders as a headless page. The following is an example of how this property can be used:

Example value for devtools_page

```
{
  ...
```

```
  "devtools_page": "components/devtools/devtools.html",
  ...
}
```

> **Note** Refer to the *Devtools Pages* chapter for details on setting up custom developer tools interfaces.

differential_fingerprint

This property is an internal key used by the Chrome Web Store for extension update distribution. You should not set this property in any circumstance.

> **Note** The Chrome Web Store generates the property when sending out a differential extension update. The property uniquely identifies only the files that changed in the new version of that extension. The key is automatically stripped out when the extension is installed.

event_rules

(Chromium browsers only) This property is deprecated; it defines rules that can modify in-flight network requests using `declarativeWebRequest` or take actions depending on the page content – all without requiring permission to read the page's content using `declarativeContent`. `declarativeWebRequest` is deprecated in favor of `declarativeNetRequest`, so this property should not be used.

externally_connectable

This property defines which foreign extensions or web pages can interact with this extension via `runtime.connect()` or `runtime.sendMessage()`. If not defined, the default behavior is to allow all foreign extensions to communicate via these methods, but to disallow all web pages from communicating via these methods. The property value is an object which can define the following:

- If the `ids` array is defined, only extensions with matching IDs are allowed to connect. This array accepts a `*` wildcard to allow all extensions.
- If the `matches` array is defined, only web pages with matching URLs will be allowed to connect.
- If the `accepts_tls_channel_id` property is set to `true`, messages will include the TLS channel ID in the outgoing message to identify the originating frame.

The following is an example of how this property can be used:

Example value for externally_connectable

```
{
  ...
  "externally_connectable": {
```

```
    "ids": [
      "allowmeabcdefabcdefabcdefabcdefa"
    ],
    "matches": [
      "https://*.example.com/*"
    ],
    "accepts_tls_channel_id": true
  },
  ...
}
```

file_browser_handlers

(Manifest v3 and Chrome/Chrome OS only) This property defines an array containing file browser handler objects that can extend the Chrome OS file browser. This is only applicable on Chrome OS devices. The following is an example of how this property can be used:

Example value for file_browser_handlers

```
{
  ...
  "file_browser_handlers": [
    {
      "id": "upload",
      "default_title": "Save File",
      "file_filters": [
        "filesystem:*.*",
      ]
    }
  ],
  ...
}
```

Note The Chrome OS file browser handler is out of the scope of this book. For documentation, refer to https://developer.chrome.com/docs/extensions/reference/fileBrowserHandler/.

file_system_provider_capabilities

(Manifest v3 and Chrome/Chrome OS only) This property defines an object containing properties that govern how the extension can interact with the host device's filesystem via the File System Provider API. This is only applicable on Chrome OS devices. The following is an example of how this property can be used:

Example value for file_system_provider_capabilities

```
{
  ...
  "file_system_provider_capabilities": {
    "configurable": true,
    "watchable": false,
```

```
      "multiple_mounts": true,
      "source": "network"
    },
    ...
  }
```

Note The Chrome OS file manager and filesystem are out of the scope of this book. For documentation, refer to https://developer.chrome.com/docs/extensions/reference/fileSystemProvider/.

homepage_url

This property provides an extension homepage URL for display in the browser. The following is an example of how this property can be used:

Example value for homepage_url

```
{
  ...
  "homepage_url": "https://www.buildingbrowserextensions.com"
  ...
}
```

Note If the `developer.url` property is defined, it will override this value. This is not supported in Chrome or Safari.

host_permissions

(Manifest v3 only) This property defines the host match patterns that the extension requires to run. If the web page matches one or more patterns in this list, the extension will have the ability to read or modify host data, such as `cookies`, `webRequest`, and `tabs.executeScript`. The following is an example of how this property can be used:

Example value for host_permissions

```
{
  ...
  "host_permissions": [
    "*://developer.mozilla.org/*",
    "*://developer.chrome.com/*"
  ],
  ...
}
```

Importantly, the `host_permissions` values are unrelated to the `content_scripts` pattern matchers. Pattern matchers that control which pages should have content scripts injected are defined inside the `content_scripts` property.

Note Refer to the *Permissions* chapter for details on extension permissions.

icons

This property defines an object containing values which dictate the extension's primary icon. This icon is used both during installation as well as in various extension marketplaces. At minimum, you should define a 128x128 raster image (JPEG, PNG, BMP, or ICO image). If you wish to support all marketplaces and browsers, you should define four PNG images with dimensions 16x16, 32x32, 48x48, and 128x128.

The following is an example of how this property can be used:

Example value for icons

```
{
  ...
  "icons": {
    "16": "assets/icons/icon-16.png",
    "32": " assets/icons/icon-32.png",
    "48": " assets/icons/icon-48.png",
    "128": " assets/icons/icon-128.png"
  },
  ...
}
```

Note SVG files are not supported for this property.

incognito

This property allows you to define how the extension can interact with private browsing windows. The following is an example of how this property can be used:

Example value for incognito

```
{
  ...
  "incognito": "spanning"
  ...
}
```

The `incognito` property can have one of three values:

- `spanning` allows the extension to treat both private and non-private pages in the same manner. It can differentiate between the two via the `chrome.windows.getLastFocused()` method. This value is the default.
- `split` will partition the extension into two separate parts: one part handles private pages, one part handles non-private pages. These

parts behave as two separate extension instances.

- `not_allowed` disables the extension on private browsing.

Note Support for this property is fragmented between browsers, refer to https://developer.chrome.com/docs/extensions/mv2/manifest/incognito/ and https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/manifest.json/incognito for details.

key

(Chromium browsers only) This property explicitly defines for the browser what the 32-character extension ID should be. If `key` is not provided, the browser will automatically generate it for you.

Example value for key

```
{
  ...
  "key": "hdokiejnpivrijdhajhdlcegeplioahd"
  ...
}
```

Note This is only needed in the rare circumstance where you wish to explicitly define the extension ID for local development at load time. This value is not used when deploying to a web store.

manifest_version

This property defines which indicates to the browser how the manifest should be interpreted. As described earlier in the chapter, this integer has significant implications for how the overall manifest file must be structured. The following is an example of how this property can be used:

Example value for manifest_version

```
{
  ...
  "manifest_version": 3
  ...
}
```

Note Some browsers like Firefox are still in the process of rolling out support for manifest v3, but other browsers like Chrome are actively phasing out support for manifest v2. Your application will need to generate multiple `manifest.json` files to support multiple marketplaces.

minimum_chrome_version

(Chromium browsers only) This property defines the minimum version of Chrome required for the extension. Non-Chromium browsers will ignore

this property. The following is an example of how this property can be used:

Example value for minimum_chrome_version

```
{
  ...
  "minimum_chrome_version": "90"
  ...
}
```

nacl_modules

This property was used for Native Client support (NaCl), its use is deprecated.

> **Note** Native Client was a sandbox for running compiled C and C++ code in the browser efficiently and securely, independent of the user's operating system. It was deprecated in 2020 and support ended in June 2021.

name

This property defines the formal name of the extension. This value is displayed in the browser as well as in the extension marketplaces. This name should not exceed 45 characters. The following is an example of how this property can be used:

Example value for name

```
{
  ...
  "name": "Building Browser Extensions",
  ...
}
```

oauth2

(Chromium browsers only) This property is used to register your extension as an OAuth2 client. Its value is an object with the OAuth2 client ID and scopes:

Example value for optional_permissions

```
{
  ...
  "oauth2": {
    "client_id": "oAuthClientID.apps.googleusercontent.com",
    "scopes": [
      "https://www.googleapis.com/auth/contacts.readonly"
    ],
  },
  ...
```

```
}
```

Note Refer to the *Extension and Browser APIs* chapter for details on setting up authentication for your extension.

offline_enabled

This property is deprecated and should not be used. In legacy codebases, it indicated if the extension was expected to work offline.

omnibox

This property enables the browser's native omnibox interface by defining its entry keyword. The value is an object containing a single `keyword` property. The following is an example of how this property can be used:

Example value for omnibox

```
{
  ...
  "omnibox": {
    "keyword": "bbx",
  },
  ...
}
```

Suppose a user has an extension installed with the omnibox configuration shown above. The omnibox interface works as follows:

1. The user focuses on the empty browser address bar.
2. The user types `bbx` followed by a space.
3. The browser opens the omnibox interface (Figure 5-10).

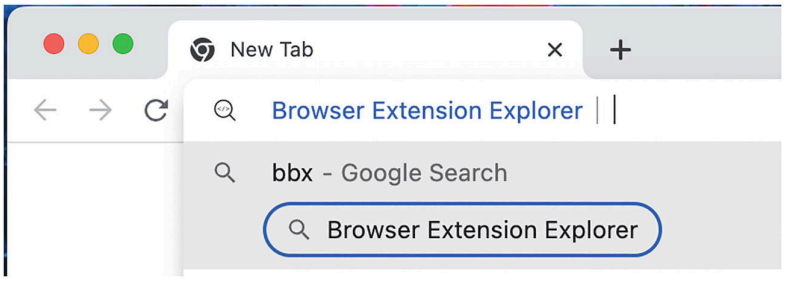

Figure 5-10 The omnibox interface

Note The extension can control the omnibox behavior with the Omnibox API. This is covered in depth in the *Extension and Browser APIs* chapter.

optional_host_permissions

(Manifest v3 and Chromium only) This property defines the host match patterns that the extension does not explicitly require to run, but that the

user can opt into. The match pattern behavior and syntax are identical to `host_permissions`. The permission granting user interface is identical to `optional_permissions`.

optional_permissions

This property defines the permissions that the extension does not require to run correctly, but that it can prompt the user for access at runtime. This property is often used to add extension permissions in subsequent releases without requiring that all users grant access to a new permission. Not all permission types are eligible to appear in this array. The following is an example of how this property can be used:

Example value for optional_permissions

```
{
  ...
  "optional_permissions": [
    "cookies",
    "history",
    "notifications"
  ],
  ...
}
```

Note Refer to the *Permissions* chapter for details on extension permissions.

options_page

This is a deprecated property. Use `options_ui` instead.

options_ui

This property tells the browser where to locate the HTML file that should render as the options page when opened either programmatically, by URL, or via the toolbar context menu. It is an object that typically only contains a single `page` property, but also supports additional properties in some browsers. The following is an example of how this property can be used:

Example value for options_ui

```
{
  ...
  "options_ui": {
    "page": "components/options/options.html"
  },
  ...
}
```

browser_style

(Firefox only) This property is a boolean that tells the browser if it should inject a stylesheet in the options page to style it consistently with the browser. These stylesheets can be viewed in any Firefox browser at `chrome://browser/content/extension.css`, or `chrome://browser/content/extension-mac.css` on macOS. The property defaults to `false`. The following is an example of how this property can be used:

Example value for browser_style

```
{
  ...
  "options_ui": {
    "page": "components/options/options.html",
    "browser_style": true,
  },
  ...
}
```

open_in_tab

(Firefox only) This property is a boolean that tells the browser if it should open the options page in a normal browser tab, rather than in an embedded options page (Figure 5-11). The property defaults to `true`.

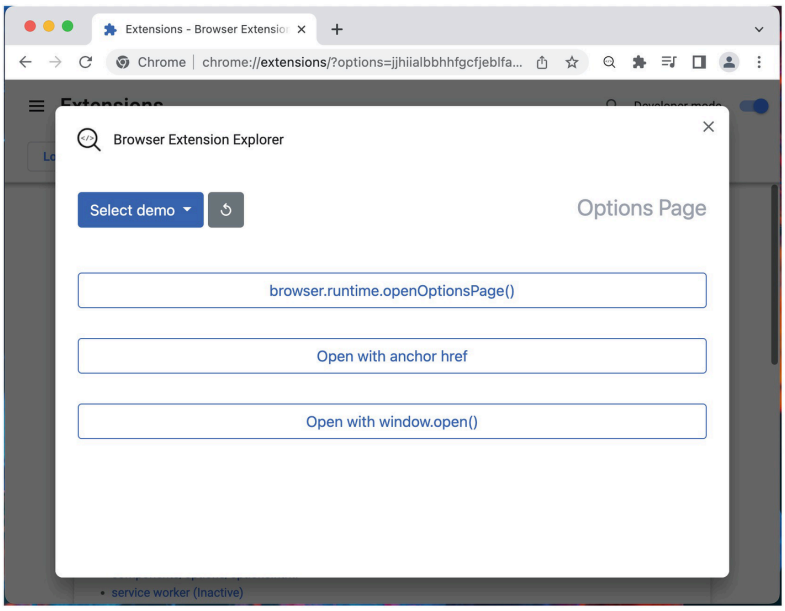

Figure 5-11 An embedded options interface

The following is an example of how this property can be used:

Example value for options_ui

```
{
  ...
```

```
  "options_ui": {
    "page": "components/options/options.html",
    "open_in_tab": false,
  },
  ...
}
```

page_action

(Manifest v2 only) The `page_action` property was replaced by `action` in manifest v3. The property values and behaviors are effectively identical. If using manifest v2, refer to the `action` property for details on how to assign a value for this property.

> **Note** This property is an artifact of extension tooling for older browser versions. `browser_action` and `page_action` are redundant in all modern browsers. As a result, these properties have been merged into the `action` property, which is identical in format and behavior.

permissions

This property defines the permissions that the extension requires to run correctly. All permission types are eligible to appear in this array. Upon installation and updates, all additions to this array will require that users explicitly grant access to the newly added permission in a popup dialog. The following is an example of how this property can be used:

Example value for permissions

```
{
  ...
  "permissions": [
    "activeTab",
    "declarativeNetRequest",
    "geolocation"
  ],
  ...
}
```

In manifest v2, the permissions array also included URL pattern matchers that are now defined in the `host_permissions` property. Modern extensions should now be placing URL pattern matchers only in the `host_permissions` or `optional_host_permissions` properties.

> **Note** Refer to the *Permissions* chapter for details on extension permissions.

platforms

This property was used for Native Client support (NaCl), its use is deprecated.

Note Native Client was a sandbox for running compiled C and C++ code in the browser efficiently and securely, independent of the user's operating system. It was deprecated in 2020 and support ended in June 2021.

replacement_web_app

This property was used as part of Google's supported transition from Chrome Apps in 2016. Its purpose was so that Chrome Apps, which shared APIs with web extensions, could direct their users to a replacement Progressive Web Application via a `installReplacementWebApp()` method. No modern extensions will have a use for this property.

Note Read more about the transition from Chrome Apps here: https://developer.chrome.com/docs/apps/migration/.

requirements

(Chromium browsers only) This property defines the browser technologies required by the extension. If a user's device does not meet the requirements, the Chrome Web Store will inform them that their device lacks the needed technology to run the extension. Currently, the only actively supported property is "3D." The following is an example of how this property can be used:

Example value for requirements

```
{
  ...
  "requirements": {
    "3D": {
      "features": [
        "webgl"
      ]
    }
  },
  ...
}
```

sandbox

(Chromium browsers only) This property defines which pages should render in sandbox mode. Its value is an object with a single `page` array containing paths to each file that should render in sandbox mode. The following is an example of how this property can be used:

Example value for sandbox

```
{
  ...
  "sandbox": {
    "pages": [
```

```
        "components/popup/popup.html",
        "components/options/options.html",
      ]
    },
    ...
}
```

Note For additional details on sandboxing pages, refer to the *Extension Architecture* chapter.

short_name

This property defines the secondary name of the extension that will be used in contexts where the `name` property is too long. This name should not exceed 12 characters. If this is not provided, the browser will simply truncate the `name` property. The following is an example of how this property can be used:

Example value for short_name

```
{
  ...
  "short_name": "BBX",
  ...
}
```

storage

This property specifies the schema file for managed storage, which is involved with the `storage.managed` API. The following is an example of how this property can be used:

Example value for storage

```
{
  ...
  "storage": {
    "managed_schema": "schema.json"
  },
  ...
}
```

Note Browsers handle managed storage differently. Refer to the *Extension and Browser APIs* chapter for more details.

system_indicator

This property is deprecated and should not be used.

Note Refer to
https://bugs.chromium.org/p/chromium/issues/detail?id=142450

for a history of this property.

tts_engine

(Chromium browsers only) This property is used to declare all the voices and voice configurations the extension wishes to use for the browser's text-to-speech engine. The following is an example of how this property can be used:

Example value for tts_engine

```
{
  ...
  "tts_engine": {
    "voices": [
      {
        "voice_name": "Alice",
        "lang": "en-US",
        "event_types": ["start", "marker", "end"]
      },
      {
        "voice_name": "Pat",
        "lang": "en-US",
        "event_types": ["end"]
      }
    ]
  },
  ...
}
```

Note For details on Chrome's text-to-speech API, refer to https://developer.chrome.com/docs/extensions/reference/ttsEngine/.

update_url

(Chromium browsers only) This property defines the URL from which the browser should request updates. It is only used for Chrome extensions that are not hosted on the Chrome Web Store. The following is an example of how this property can be used:

Example value for update_url

```
{
  ...
  "update_url": "https://example.com/updates.xml",
  ...
}
```

Note Refer to the *Extension Development and Deployment* chapter for details on self-hosting extensions.

version

This property defines the formal version number of the extension. It is used to both uniquely identify different releases of your extension, as well as to determine ordinality of those releases.

Browsers set different requirements for this value; for example, the Chrome version validator is more restrictive than Firefox. For simplicity and cross-browser compatibility, it is strongly recommended that you conform to the semantic versioning standards described at **https://semver.org/**. The following is an example of how this property can be used:

Example value for version

```
{
  ...
  "version": "1.5.0"
  ...
}
```

Note Using the `MAJOR.MINOR`, `MAJOR.MINOR.PATCH`, or `MAJOR.MINOR.PATCH.BUILD` are all acceptable version formats that all browsers will accept.

version_name

This property defines a descriptor for the extension version. It allows you to add in handles like `beta` or `rc1` that offer additional information about the release but do not conform to the browser's extension version validators. The following is an example of how this property can be used:

Example value for version_name

```
{
  ...
  "version_name": "1.5.0 beta"
  ...
}
```

web_accessible_resources

This property defines which files in the extension can be accessed outside the extension, either the web page or another extension. Any files that are part of the extension are eligible to be listed here, but this property is intended to allow for JS, CSS, HTML, and image files in the extension to be added into the page or a foreign extension. The property is used in both manifest v2 and v3, but the manifest v3 specification adds extra controls for managing an access control list for each resource.

Manifest v2

The manifest v2 version of `web_accessible_resources` is very simple: simply an array of pattern strings. If a resource matched this pattern string, web pages or remote extensions were granted read access. The following is an example of how this property can be used:

Example value for manifest v2 web_accessible_resources

```
{
  ...
  "web_accessible_resources": [
    "scripts/widget.js",
    "assets/images/*"
  ]
  ...
}
```

Manifest v3

Manifest v3 extends this property to allow for control of which origins or extensions have access to certain assets, as well as the option to enable dynamic resource URLs. The new format is an array of objects with the following properties:

- `resources` is the list of patterns that match files in the extension. This is equivalent to the full `web_accessible_resources` property in manifest v2. This property is required
- `matches` and `extension_ids` control where the resources list is accessible. Exactly one of these properties must be defined. `matches` is an array of patterns that match web page origins eligible to access the resource list. `extension_ids` is an array of extension IDs eligible to access the resource list.
- `use_dynamic_url` is a boolean that, when `true`, will instruct the browser to rotate the URL of the resources each session. This property defaults to `false`. Because extension IDs are static throughout the lifetime of that extension, any resource by default will have the same URL between releases. Enabling this property makes the URL of that resource unpredictable.

Summary

In this chapter, you were guided through what a manifest does and how to create one. The chapter took you through all possible manifest properties, what they do, how they should be defined, what APIs they work with, and which browsers can use them. When you come across a manifest property, you should be able to refer to the relevant section in this chapter to quickly decipher what that property is doing.

The next chapter will cover the transition from manifest v2 to v3. It explores the differences between the two versions and how it will affect the world of browser extensions.